CS240: Lab 10 (30 pts)

TAs: Darshan

March 30, 2026

General Instructions

1. Welcome to the **graded Inlab** on RNNs. In this Inlab, you will work on three tasks in total. Each task is marked with stars (★) to indicate their difficulty and all the tasks are graded.
2. IMPORTANT NOTE: DO NOT TRY TO GAME THE SYSTEM. For each task, we have several checks and alternative tasks to ensure that you have solved the question as we asked. Any deviations will attract harsh penalties.
3. In each task, you are allowed to modify only the **model.py** file. We will not entertain any regrading requests if the autograder fails. However, for your entertainment, you can tinker around the values for learning rate, dataset size, train-test split etc. During final grading, we will only use model.py file.
4. For all the problems, the sequence length is fixed to $N = 25$, and input values have the constraint $0 \leq x_i \leq 9$.

Task 1: Finding bugs (6 pts) [★]

In this task, you are given the reference solution for Task 1 from the ungraded Outlab (the task where you were asked to implement RNNs from scratch). Unfortunately, the TA responsible for writing this code did it at 3 am with caffeine running through his veins. As a result, he has inadvertently introduced several bugs and is now unable to find them. He says that while some of these bugs are breaking the code, others are silently sabotaging model's learning. Your mission is to identify and fix all the errors so that the TA is saved from the wrath of the head TAs. Note that your solution code must not only run without errors but also demonstrate correct loss reduction during training. [*Hint: There are exactly six bugs in the code. These bugs can be modification, insertion or deletion errors.*]

Task 2: Reimagining RNNs (10 pts) [★★]

In this task, you are presented with specific sequence-to-sequence problems. Your objective is to design alternative RNN formulations that would outperform a vanilla RNN. Improvement can either be: enabling the RNN to solve a problem that a standard version cannot, reducing the total parameter count, or significantly boosting computational speed. Unlike the questions in Quarterly Quiz-2, here you are not required to calculate the final values of the weights. Instead, focus on proposing structural modifications to the RNN equations tailored to each scenario. Once the architecture is correctly defined, the training process will handle the optimization.

For example, recall Task-3 from the Outlab, where, to make RNNs faster, we eliminated the non-linear activation between hidden states. This allowed us to transform the recurrence into a linear system that we could compute as an extremely fast convolution operation.

Problem 1: Incrementing array values (4 pts)

Your input is a sequence of N numbers $\mathbf{X} = [x_1, x_2, \ldots, x_N]$ and your goal is to produce an output sequence $\mathbf{Y} = [y_1, y_2, \ldots, y_N]$, such that $y_i = 2 * x_i + K$. Here, K is a hyperparameter. For example:

```
Input (X):  [1, 2, 3, 4, 5], K = 3
Output (Y): [5, 7, 9, 11, 13]
```

[*Hint: Does the output at step t depend on any information from step $t-1$? If not, how does this change parameter requirements?*]

Problem 2: Windowed Summation (6 pts)

Your input is a sequence of N numbers $\mathbf{X} = [x_1, x_2, \ldots, x_N]$ and your goal is to produce an output sequence $\mathbf{Y} = [y_1, y_2, \ldots, y_N]$, such that

$$y_i = \sum_{j=\max(0,i-K)}^{\max(i+K,N)} x_j$$

where K is a hyperparameter. For example:

```
Input (X):  [1, 2, 3, 4, 5], K = 1
Output (Y): [3, 6, 9, 12, 9]
```

[*Hint: This problem requires "Information from the Future". Recall that a standard causal recurrence $h_t = f(h_{t-1}, x_t)$ cannot access x_{i+K}. How can you bridge this gap?*]

Task 3: Attention in RNNs (14 pts) [★★★]

In this task, you will implement an Encoder-Decoder RNN using a modified Attention mechanism [1]. Unlike the standard RNN which compresses the entire history into a single fixed-length hidden vector, the attention mechanism allows the model to dynamically look at specific parts of the input and create its own context for each output step. This breakthrough was the precursor to the now popular Transformers architecture. For this task, you will be solving Problem 2 from Task 2.

To be specific, you will be implementing the following:

1. The Encoder Architecture

The Encoder is responsible for transforming the input sequence $\mathbf{X} = [x_1, x_2, \ldots, x_T]$ where $x_i \in \mathcal{V}$ (here $\mathcal{V}$ denotes the vocabulary) into a sequence of meaningful hidden representations $\mathbf{H} = [h_1, h_2, \ldots h_T]$, where $h_i \in \mathbb{R}^d$. Here, you will implement the same architecture as in Task 1. That is,

$$h_i = \tanh(Wx_i + Uh_{i-1} + b)$$

Where W and U are learnable projection matrices and b is the bias.

2. The Attention Mechanism

The attention mechanism computes a context vector c_t by determining which hidden states in $\mathbf{H}$ are most relevant to the current output step t.

- **Energy Scores:** For the current timestep t, we use the encoder state h_t as the query. For each vector h_i in $\mathbf{H}$, compute the scalar energy score $e_{t,i}$ as:

  $$e_{t,i} = v_a^\top \tanh(W_a h_t + U_a h_i)$$

  Here, h_t acts as the positional anchor, and v_a, W_a, U_a are learnable alignment parameters.

- **Attention Weights:** Normalize the scores using the Softmax function:

  $$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T} \exp(e_{t,j})}$$

- **Context Vector:** The context vector c_t is the weighted sum of all encoder representations:

  $$c_t = \sum_{i=1}^{T} \alpha_{t,i} h_i$$

3. The Decoder Architecture

The Decoder generates the output sequence $\mathbf{Y}$ by iterating through the encoder output. For each step t, it updates its internal state s_t using the context vector c_t generated by the attention module:

$$s_t = \tanh(W_s s_{t-1} + W_c c_t + b)$$

The final output probability distribution for step t (the predicted sum) is calculated by projecting the concatenated decoder state and context vector:

$$P(y_t \mid \mathbf{X}) = \text{Softmax}(V s_t + M c_t + c)$$

Here, $W_a, U_a, v_a, W_s, W_c, V$, and M are all learnable weight parameters.

References

[1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate, 2016.